

General Disclosure

Model Used: GPT-5.6 Sol, thinking mode set to high.

Date(s) Used: 09/27/26

Code Attribution

Prompt 1:

Write a C++ program that reads a line of text and extracts a single valid IPv4 address embedded anywhere in that text. Only digits, periods (.), and colons (:) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar in full — no partial matches, no truncating to find a valid piece inside a longer run.

Here is the given function prototype:

C++

```
// Returns true if a valid address was found, false otherwise.  
// On success: outAddress holds the 32-bit value,  
// and outPort holds the port number, or -1 if no port was present.  
// On failure: outAddress is set to 0 and outPort is set to -1.  
bool extractIPv4(const std::string& str, unsigned long& outAddress,  
int& outPort);
```

Prompt 2:

This code runs. We are using some unauthorized modules like stoul, but let's ignore that for now and see if this works first and we can fix that later. To test, we need to make a user terminal interface to input some values. Here is a sample output:

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now  
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:  
none)  
Enter a string (or 'END' to quit): server=10.0.0.255:8080end  
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port:  
8080)  
Enter a string (or 'END' to quit): 192a168.1.1.1  
Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port:  
none)  
Enter a string (or 'END' to quit): 192.168.1.1.  
Invalid input: no valid IPv4 address found  
Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused  
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:  
none)  
Enter a string (or 'END' to quit): 192.168.01.1
```

```
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 12.34.56
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): no number here
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): END
Program terminated.
```

Prompt 3:

This mostly works. The only issue I found is the output with the 192.168.1.1. test case. Here is the output:

```
Enter a string (or 'END' to quit): 192.168.1.1.
```

```
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
```

The expected output is "Invalid input: no valid IPv4 address found"

In the code, we catch most unwanted characters, but we do not check for leading/trailing colons and periods. Those are technically wanted characters, but they are not wanted at the beginning or end of the inputs. Lets write code to fix this issue.

Prompt 4:

Great, now this works. I now need to make sure that this is meeting these requirements:

Built-in functions/libraries you may not use

- Any string-to-number conversion function: `atoi`, `atol`, `atoll`, `strtol`, `strtoul`, `strtod`, `stoi`, `stol`, `stoul`, `sscanf`, `scanf` with numeric conversions.
- Any address-parsing library function: `inet_aton`, `inet_pton`, `inet_addr`, or equivalents.
- Any regular-expression facility (`std::regex`, `POSIX regex.h`, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library.
- Standard character-classification functions (`isdigit`, etc.) are fine to use.

Other requirements

- Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation.
- Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a

colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.

- On success, print: `Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)` where N is the 32-bit decimal value and P is the port number or the literal text none.

Program requirements

- **Input loop:** `main` continuously prompts the user for input until the user enters END (case-sensitive), then prints Program terminated. and exits.
- **Extraction function:** implement `extractIPv4` exactly as prototyped above. All digit accumulation must be done by hand.
- **Display:** `main` receives the result from `extractIPv4` and formats the output exactly as specified above.

This is going back to what I was saying earlier about how we are using stoul. Lets go ahead and write code to replace these and any other issues you see with the code.

All parts of the code were written by AI. I would review it, but I let the AI fix the mistakes for me once I tested everything.

Verification Statement

I verify that I understand all lines of code. The code has been tested and works as intended. There are no known issues, bugs, or limitations with the code.